PROJECT REPORT

Compiler Design —

Syntax Parse Implementation using Recursive Decent parsing

Module 2 & 3.

Name - Priyanshu Kumar

Course - B.Tech CSE (AI & ML)

Semester - 6th

Section - 3CSE1

Roll no - CS-23411269

INDEX

1. Introduction
2. Grammar
3. Recursive function
4. Parsing process
5. Evaluating logic
6. Program code
7. Test Cases
8. Challenges faced
9. conclusion

```
        pos += 1
        value = value * T()
    return value

def E():
    global pos, expr
    value = T()
    while pos < len(expr) and expr[pos] == '+':
        pos += 1
        value = value + E()
    return value

expr = input("Enter Expression: ")
pos = 0
result = E()

If pos != len(expr):
    print("Invalid Expression")

else:
    print("Result =", result)
```

7. Test Cases

The following test Cases were used:

1. 2*3+4 → Result = 10
2. (2+3)*4 → Result = 20
3. 5+2*3 → Result = 11
4. 2+3+1 → Result = 6

5. Evaluation Logic

The Parse not only checks the syntax but also evaluates the expression.

- Digits are converted into integers
- Multiplication is performed first
- Addition is performed after multiplication
- Parentheses are evaluated after.

(EX):-

Input: 2*3+4

output: 10

6. Program Code

```
expr = ""
pos = 0

def F():
    global pos, expr
    if pos < len(expr) and expr[pos] = '(':
        pos += 1
        value = E()
        If pos < len(expr) and expr[pos] = ')':
            pos +=1
            return value
        else:
            raise Exception("missing)")
    elif pos < len(expr) and expr[pos].isdigit
        value = int(expr[pos])
        pos += 1
        return value
    else:
        raise Exception("Invalid character")

def T():
    global pos, expr
    value = F()
    while pos < len(expr) and expr[po
        == '*':
```

3. Recursive functions

We implemented three recursive functions based on the grammer:

F() - factor

This function handles:

- If the current character is C, then it calls E() expects)
- If the character is a digit, it converts into an integer

This represents the smallest unit of the expression.

E() - Expression

This function handles addition:

- It first calls T()
- If + is found, it recursively calls E() and adds the values

4. Parsing process

The input expression is stored in a starting variable. A global variable pos is used to track the current possition.

Steps:

1. The parsing starts from function E()
2. E() calls T()
3. T() calls F()
4. Based on operators, function are called recursively.
5. The final result is calculated during parsing.

1. Introduction

→ In compiler design, Parsing is an important phase where the input string is analyzed based on a given grammar. The main goal of parsing is to check whether the input expression is valid and to understand its structure.

In this module, we implemented a Recursive Descent parse for simple arithematic expression. Recursive decent parsing is a top-down parsing technique in which each non-terminal of the grammer is represented by a separate function. The parser reads the input expression from left to right and evaluates it during parsing. This module helps in understanding the basic concepts of parsing and lays the function for building a compiler.

2. Grammar

The grammar used in this module is

$E \rightarrow T+E \mid T$

$T \rightarrow F*T \mid F$

$F \rightarrow (E) \mid digit$

where;

- E = Expression
- T = Term
- F = Factor

This grammer follows operator precedence:

- * has higher precedence than +
- Parenthese () have the highest precedence

8. Challenges faced

During the implementation, the following challenges were faced:

- Understanding the grammer
- Managing recursive
- Handling parenthese correctly.
- Debugging position tracking errors.

These challenges were solved through testing and debugging.

9. Conclussion

In this modules, we successfully implemented a Recursive Decent parser that can parse and evaluate arithematic expressions.

This project improved our understanding of parsing techniques, recursion, and compiler design concepts. it also provide a strong base for building more advance compiler modules in the features.